FOODIEE

Food Ordering Web Application

Project Documentation

Technologies

Java • Spring Boot • JSP/Servlets • HTML5 • CSS3 • JavaScript • MySQL

Prepared for Academic / GitHub Project Submission

1. Abstract

Foodiee is a web-based food ordering application designed to provide a convenient platform for customers to browse restaurants and food items, manage a shopping cart, place orders, and track order status. The system also provides restaurant and administrative functionality for managing restaurants, menus, customers, orders, and related transaction data.

The application is developed using Java and Spring Boot for the backend, JSP/Servlets for web interaction, HTML5, CSS3 and JavaScript for the frontend, and MySQL for persistent data storage. The project demonstrates database-driven application development, CRUD operations, authentication-oriented workflows, order processing, and modular web application design.

2. Introduction

Online food ordering has become an important part of modern digital services. Customers expect to discover food options, add items to a cart, provide delivery information, place orders, and monitor order progress through a simple web interface. Foodiee addresses these requirements through a centralized food ordering system.

The application connects customers with restaurant and menu information while maintaining order and transaction records in a relational database. It is suitable as an academic project demonstrating Java web development, Spring Boot, JSP/Servlets, JavaScript and MySQL integration.

3. Problem Statement

Traditional food ordering through phone calls or in-person visits can be time-consuming and may lead to errors in menu selection, order details, pricing, and order status communication. A centralized web application is required to simplify food discovery, ordering, data management, and order tracking.

4. Objectives

• Provide an easy-to-use online food ordering interface.

• Allow customers to browse restaurants and food items.

• Provide shopping cart and checkout functionality.

• Maintain customer, restaurant, menu and order information.

• Support CRUD operations for application data.

• Store application and transaction data in MySQL.

• Provide order status and tracking functionality.

• Provide administrative management features.

5. Scope of the Project

The project covers customer-facing food browsing and ordering, restaurant/menu data management, cart and checkout workflows, order processing and administrative management. The system can be extended later with online payment gateway integration, delivery partner management, notifications, reviews and mobile applications.

6. Existing System

In a basic traditional ordering process, customers may depend on phone calls, printed menus or direct visits. Such approaches provide limited automation and can make it difficult to maintain accurate order history, customer information and real-time order status.

7. Proposed System

Foodiee provides a centralized web-based solution where users can browse available food, select items, maintain a cart, complete checkout and follow order status. Administrative functionality helps maintain application data and monitor operations.

8. Functional Requirements

• Customer registration/login-oriented workflow.

• Restaurant listing and restaurant information.

• Food/menu listing and item selection.

• Add, update and remove cart items.

• Checkout and order placement.

• Order history and order tracking.

• Customer data management.

• Restaurant and food-item CRUD operations.

• Order management for administrators.

• Database storage for application and transaction information.

9. Non-Functional Requirements

• Usability: interfaces should be simple and understandable.

• Performance: common database operations should respond efficiently.

• Reliability: order and customer data should be stored consistently.

• Maintainability: modules should be organized for future enhancement.

• Security: access to administrative functions should be restricted appropriately.

• Scalability: the architecture should allow additional restaurants, menu items and users.

10. Technology Stack

Layer	Technology
Programming Language	Java
Backend Framework	Spring Boot

Web Layer	JSP / Servlets
Frontend	HTML5, CSS3, JavaScript
Database	MySQL
Development	Eclipse / compatible Java IDE
Version Control	Git / GitHub

11. System Modules

11.1 Customer Module

The customer module supports customer-oriented activities such as browsing restaurants and menus, selecting food items, managing the shopping cart, checking out, placing orders and viewing order information.

11.2 Restaurant Module

The restaurant module represents restaurant information and menu items. Restaurant-related data can be maintained through CRUD operations.

11.3 Menu / Food Item Module

This module manages food items, descriptions, prices and associated restaurant information. It enables menu data to be presented to customers.

11.4 Shopping Cart Module

The shopping cart temporarily maintains selected food items before checkout. Customers can add items, change quantities and remove items.

11.5 Checkout and Order Module

During checkout, the selected items and customer information are used to create an order. Order details are persisted in the database for subsequent processing and tracking.

11.6 Payment Module

The application includes a payment-related workflow as part of the ordering process. Actual production payment-gateway integration can be added as a future enhancement if not implemented in the current version.

11.7 Admin Module

The admin module provides management capabilities for customers, restaurants, food items and orders. CRUD operations allow authorized administrators to maintain application records.

12. Database Design

MySQL is used as the relational database. The application requires tables for core entities and transaction data. A typical logical design includes the following entities; the exact column names should match the implemented project database.

• Customer — stores customer identity and account information.

• Restaurant — stores restaurant details.

• Food/Menu Item — stores food item, price and restaurant association.

• Order — stores order-level information such as customer, date, amount and status.

• Order Item — stores individual food items and quantities belonging to an order.

• Payment — stores payment-related transaction information where applicable.

13. Entity Relationships

• A customer can place multiple orders.

• A restaurant can have multiple food/menu items.

• An order can contain multiple order items.

• Each order item refers to a selected food/menu item.

• An order may have an associated payment record.

14. Application Workflow

• User opens the Foodiee web application.

• Customer browses available restaurants and menus.

• Customer selects food items and adds them to the cart.

• Customer reviews or updates cart quantities.

• Customer proceeds to checkout.

• Order information is validated and stored.

• Payment-related processing is performed according to the implemented workflow.

• Order status is updated and can be tracked.

• Administrator manages restaurants, menu items and orders.

15. CRUD Operations

CRUD stands for Create, Read, Update and Delete. Foodiee uses CRUD operations for database-driven management of customers, restaurants, food items and orders.

• Create: add new customers, restaurants, menu items and records.

• Read: retrieve and display records from MySQL.

• Update: modify existing restaurant, menu, customer or order information.

• Delete: remove records when permitted by the application's business rules.

16. User Interface

The frontend uses HTML5, CSS3, JSP and JavaScript to provide web pages and interactive behavior. Typical screens include home/restaurant listing, menu, cart, checkout, order details, login/registration and administrative management pages.

17. Testing

Testing should verify individual modules as well as complete ordering workflows.

Test Area	Expected Result
Restaurant listing	Available restaurant information is displayed.
Menu display	Food items and prices are displayed correctly.
Cart	Items can be added, updated and removed.
Checkout	Valid order information is accepted.
Order creation	Order record is stored correctly.
CRUD	Authorized create/read/update/delete operations work correctly.
Database	Records are inserted and retrieved consistently.
Order tracking	Current order status is displayed according to application data.

18. Advantages

• Centralized online food ordering.

• Reduced dependence on manual ordering.

• Structured management of restaurant and menu information.

• Database-backed order history.

• Shopping cart and checkout workflow.

• Modular architecture suitable for academic demonstration and further development.

19. Limitations

• Production-scale deployment would require stronger security and performance hardening.

• Real payment gateway integration may require additional configuration.

• Real-time delivery tracking requires external location/delivery services.

• Mobile-specific applications are not included in the current web application scope.

20. Future Scope

• Integrate secure payment gateways.

• Develop Android/iOS applications.

• Add restaurant ratings and customer reviews.

• Add coupon and promotional systems.

• Implement real-time delivery tracking.

• Add email/SMS/push notifications.

• Introduce analytics dashboards for restaurants and administrators.

• Improve authentication, authorization and security controls.

• Deploy the application to a cloud platform.

21. Conclusion

Foodiee demonstrates the design and development of a database-driven food ordering web application using Java, Spring Boot, JSP/Servlets, HTML5, CSS3, JavaScript and MySQL. The project combines customer ordering functionality with restaurant, menu, order and administrative management. It provides a strong academic demonstration of web application development, CRUD operations, relational database design and end-to-end ordering workflows.

22. GitHub Repository Documentation

Recommended repository contents:

• Source code organized into appropriate backend, web and configuration packages.

• Database SQL script or database setup instructions.

• README.md containing project overview, technologies and run instructions.

• Project documentation PDF.

• Application screenshots.

• Sample configuration/template files without exposing passwords or secrets.

Suggested README Project Description

Foodiee — Food Ordering Web Application

A database-driven food ordering web application developed using Java, Spring Boot, JSP/Servlets, HTML5, CSS3, JavaScript and MySQL. The application includes customer, restaurant, menu, shopping cart, checkout, payment-related and order tracking modules, together with administrative CRUD management.

23. How to Run the Project

• Install Java/JDK compatible with the project.

• Install MySQL Server and create the required database.

• Import the project's SQL schema/data script if available.

• Open the project in the configured Java IDE or build tool.

• Update database connection properties with local MySQL credentials.

• Build the Spring Boot application and resolve required dependencies.

• Start the application using the project's configured run method.

• Open the application URL in a web browser.

• Test customer ordering and administrative functions.

24. References

• Java documentation and standard Java programming references.

• Spring Boot framework documentation.

• JSP/Servlet web development references.

• MySQL database documentation.

• HTML5, CSS3 and JavaScript web development references.

End of Project Documentation